

Decompose, then recompose?

Restoring and branching coding agents at natural stalls: exp48 results, and the study I'm running next

Siddharth Srinivasan · UC Berkeley · independent research · October 2026

TL;DR

- I built a harness that records a coding agent's full state every turn (conversation history, workspace files, runtime, tool outputs), so any checkpoint can be restored exactly and branched.
- From 48 natural stalls I ran **889 paired branches**: 18 interventions against plainly letting the agent continue (DeepSeek V4.1 Flash and GPT-6 Luna, hidden-test grading, 40k tokens per branch).
- **Nothing beat continuing** (34% of branches solved). Rewriting the history as a free-form summary cut the solve rate by 10 points.
- The null was mostly about headroom: in 14 of the 48 stalls nothing in any arm solved the task, and only 11 stalls were recoverable at all.

WHAT I'M CHANGING NOW

- **Only stalls with headroom.** Terminal-Bench 2.x tasks the model solves 10–50% of the time across repeated runs (screened with my own runs), and a stall counts only if at least 1 of 5 resamples from just before it still solves.
- **A submission loop.** When the agent says it's done it hears how many hidden tests fail and keeps working, so early wrong submissions become real stalls instead of quits.
- **A simpler, validated Ledger.** Each step of the full history is labelled (which approach it belongs to, whether its output is now stale, and the facts, claims and ideas it contains as exact quotes). Labels are checked against two human labellers. No compression into a fixed-size extract.
- **The agent recomposes itself.** At the stall it sees its full state, either labelled or raw, then picks a rewind point and what to drop. Headline question: does the decomposition help the agent recompose itself, compared with raw history?
- **8 arms, equal repeats** (about 10 per stall) instead of 19 arms with about one. They include a rewind + self-written summary baseline (the AgentRewind / Reflexion pattern) and a one-sentence hint as a positive control.
- **Process measures beyond solve rate.** Does the agent go back to the idea that just failed, how fast does it start a new one, and what does each arm cost with and without prompt caching?

Method (exp48)

- **Record every turn.** Agents run on tasks with hidden tests. Each turn checkpoints history, workspace, runtime and tool outputs, and a checkpoint is branchable only after a restore reproduces it.
- **Detect stalls live.** A token-based stall clock flags candidates on unmodified runs, and an LLM judge confirms each one.
- **Measure headroom first.** Before any intervention, each stall was resampled about five times from its last progress point.
- **Branch and grade.** Each of the 48 stall states (from 25 original runs, 20 MettleBench and SWE-smith tasks) is restored into every arm and graded by hidden tests.

Results: change against continuing

Arm	Solve rate, points (95% CI)	Best partial score, points
RESETS AND ROLLBACKS		
Reset history (task prompt only)	-7.3 (-15 to -1)	-1.7
Fresh runtime, keep history	-3.1 (-8 to +2)	+0.6
Roll back workspace, keep history	-3.1 (-14 to +8)	-0.6
HISTORY REWRITES		
Free-form summary of the history	-9.8 (-19 to -1)	-9.0
Ledger (full)	-5.2 (-14 to +2)	-6.9
Ledger minus plans	-5.2 (-14 to +3)	-1.0
Ledger minus beliefs	-9.4 (-19 to -1)	-4.5
Ledger minus the model's reasoning	-17.7 (-32 to -5)	-10.0
APPROACH-BOUNDARY REMOVALS (33 STATES WITH 2+ APPROACHES)		
Remove the current approach	-4.5 (-12 to +3)	+0.1
Remove a size-matched random slice	-7.6 (-16 to -2)	-1.8
Remove with the cut shifted earlier	-10.6 (-22 to -1)	-8.1

Paired per stall state, 95% cluster-bootstrap intervals over the 25 stall origins; red = interval excludes zero. Continue solved 34%. 11 of the 18 interventions shown; none of the 18 is above zero with an interval that excludes zero.

Why the null: headroom

Ceiling 14	Recoverable 11	Early origin 23
Ceiling (14). No resample solved the task, and no branch in any arm did either.	Recoverable (11). Continuing solved 27%. Too few states to separate the arms.	Early origin (23). No progress before the stall; continuing already solved 59%.

What the results do and don't show

- **Summaries lose information that matters.** A free-form summary cut the solve rate by 10 points, and on partial score it did worse than giving the agent no history at all.
- **The Ledger arms mixed compression with removal.** Each replaced the history with a roughly 3k-token extract, and every removal shrank it further, so the ablations mostly measured how much text was removed (removing reasoning also removed a median 83% of the text). Removing labelled pieces from the full history was never tested; that is the next study.
- **Runtime and workspace rollbacks changed nothing measurable.**
- **Rewrites cost more per solve.** They used 23–29k tokens per branch against 19k for continuing (the agent re-orientes first), and \$0.11–0.32 per solve against \$0.10.

Side observations on eval quality

- DeepSeek V4.1 Flash spent about 30% of its commands on SWE-smith tasks hunting for the hidden tests or the original source, in every run and in 121 of 122 recovery branches. GPT-6 Luna did this in 1 of 10 runs.
- In 21 of the 50 SWE-smith-derived tasks I packaged, the generated bug patch left comments describing the bug, and DeepSeek noticed it was inside a generated benchmark.

Caveats: most arms ran once per state because the run was stopped early to cap spending, and stall labels come from an LLM judge (a second opinion agreed on 34 of 47 reviewed states). Full preprint, per-branch data and code excerpts: github.com/H0tP0cket/de-re-

composev1